AI-Powered Business Management SaaS

Production-Oriented Development Documentation

Project Type	Multi-Tenant SaaS
Backend	Laravel 13 / PHP 8.3+
Admin	Filament
Frontend	Next.js / TypeScript
Database	PostgreSQL (recommended)
Infrastructure	Redis / Docker / Nginx / CI/CD
AI	OpenAI API / Tool Calling / RAG

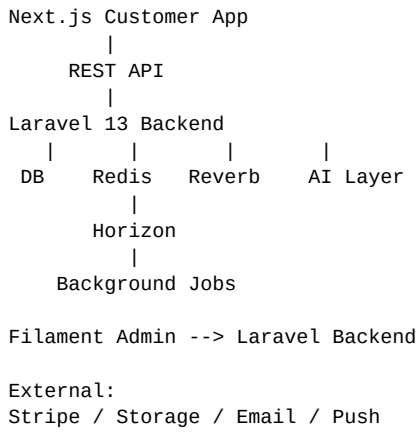
Purpose: Build a serious portfolio project covering advanced Laravel, SaaS architecture, AI integration, real-time systems, payments, testing, deployment, and scalability.

1. Project Overview

Build a multi-tenant Business Management SaaS where businesses manage customers, products, orders, payments, employees, analytics, reports, subscriptions, and operational workflows. The AI Business Assistant analyzes controlled business data and provides explanations, summaries, and recommendations.

Example: “Why did my sales decrease this month?” Laravel retrieves and calculates the relevant facts, while the AI explains the result and provides recommendations.

2. High-Level Architecture



3. Technology Stack

Layer	Technology	Purpose
Backend	Laravel 13 / PHP 8.3+	API, business logic, auth, jobs, events
Admin	Filament	Super Admin and internal management
Frontend	Next.js / TypeScript / React	Customer-facing SaaS
Database	PostgreSQL	Primary relational database
Cache / Queue	Redis	Caching, queues, locks
Queue Monitoring	Horizon	Queue monitoring
Real-time	Laravel Reverb	WebSockets/live updates
AI	OpenAI API	Assistant, tools, reports, RAG
Billing	Stripe	Subscriptions and billing
Infrastructure	Docker / Nginx / Linux	Deployment

4. User Roles

Platform: Super Admin — organizations, plans, billing, AI usage, settings, audit logs, analytics.

Business: Owner, Admin, Manager, Staff, Accountant, Viewer.

5. Multi-Tenancy

Each business is an organization/tenant. Tenant-owned records must be isolated so Organization A can never access Organization B's data.

```
organizations
organization_users
customers
products
orders
payments
reports

Most tenant-owned tables:
organization_id
```

Tenant isolation must be enforced in backend authorization and tested explicitly; frontend filtering is not a security boundary.

PHASE 0 — Project Planning & Architecture

Objective: Design the system before substantial coding.

- Define modules, system boundaries, database entities, API conventions, authentication, tenant strategy, and role/permission matrix.
- Set coding standards, Git workflow, and local environments.

Deliverables

- Architecture document
- ERD
- API conventions
- Role/permission matrix
- Git repository

PHASE 1 — Laravel Foundation

Objective: Create a clean Laravel 13 foundation.

- Configure database, Redis, mail, storage, queues, logging, testing, and Sanctum.
- Establish Actions, Services, Jobs, Events, Policies, and Support structure where useful.

PHASE 2 — Authentication & Organizations

Objective: Build the SaaS identity foundation.

- Registration creates User, Organization, and membership.
- Implement login, logout, email verification, password reset/change.
- Add organization profile, logo, timezone, currency, settings, and active-tenant context.

PHASE 3 — Roles & Permissions

Objective: Implement granular authorization.

- Create Owner, Admin, Manager, Staff, Accountant, Viewer.
- Create resource permissions such as customers.view/create/update/delete and orders.view/create/update/delete.
- Verify authentication, organization membership, permission, and resource ownership on sensitive endpoints.

PHASE 4 — Filament Admin Panel

Objective: Build internal platform and business management.

- Create dashboards for organizations, users, plans, subscriptions, AI usage, settings, audit logs, customers, products, orders, invoices, transactions, and reports.
- Practice Resources, Tables, Forms, Filters, Actions, Widgets, relation managers, notifications, and policies.

PHASE 5 — Customer Management

Objective: Build the first major business module.

- Customer fields: id, organization_id, name, email, phone, address, status, notes, timestamps.
- Implement CRUD, search, filters, pagination, profile, order history, and transactions.
- Expose REST endpoints under /api/v1/customers.

PHASE 6 — Products & Categories

Objective: Build product catalog and stock foundations.

- Products: name, SKU, category, description, price, cost, stock, status, image.
- Implement CRUD, search, filtering, sorting, pagination, image upload, stock management, indexes, and query optimization.

PHASE 7 — Orders & Business Operations

Objective: Introduce real business workflows.

- Order states: Draft, Pending, Confirmed, Processing, Completed, Cancelled.
- Create orders and items inside database transactions.
- Update stock, create transaction records, and dispatch domain events.

PHASE 8 — Payments, Invoices & Transactions

Objective: Introduce reliable financial logic.

- Implement payments, refunds, invoices, transactions, statuses, and revenue reporting.
- Practice idempotency, state transitions, webhooks, reconciliation, and audit logging.
- Never trust payment status from the frontend.

PHASE 9 — SaaS Subscriptions & Billing

Objective: Turn the application into a subscription SaaS.

- Create Free, Starter, Professional, Business, Enterprise plans.
- Define limits for users, customers, products, orders, AI requests, storage, and reports.
- Implement trials, subscription changes, cancellation, billing history, failed payments, and Stripe webhooks.

PHASE 10 — Redis, Queues & Background Jobs

Objective: Move expensive work out of HTTP requests.

- Use Redis for queues and caching.
- Create jobs for invoices, notifications, payment webhooks, reports, AI insights, and document processing.
- Practice retries, backoff, failed jobs, priorities, batching, and Horizon.

PHASE 11 — Notifications

Objective: Create a reliable notification system.

- Email and database notifications.
- Real-time notifications and optional FCM/push.
- Notify on orders, payments, subscriptions, invoices, and AI report completion.

PHASE 12 — Real-Time Features

Objective: Learn real-time Laravel architecture.

- Use Laravel Reverb and broadcasting.
- Broadcast new orders, order status changes, notification center updates, live metrics, and AI report completion.

PHASE 13 — Analytics & Reporting

Objective: Build useful business intelligence.

- Revenue, orders, customers, average order value, profit, expenses, top products, top customers, growth.
- Support common time ranges and custom ranges.
- Practice SQL aggregation, indexes, caching, and background report generation.

PHASE 14 — AI Business Assistant

Objective: Make AI useful against real business data.

- Support sales analysis, top products, inactive customers, revenue comparisons, and summaries.
- Use a secure orchestration layer; AI must not directly query the database.

PHASE 15 — AI Tool Calling

Objective: Implement structured AI-to-backend interaction.

- Tools: get_sales_summary, get_order_statistics, get_top_products, get_customer_metrics, get_revenue_comparison, get_expense_summary.
- Validate arguments and execute tools under the current organization context.
- Keep deterministic calculations in Laravel/database code.

PHASE 16 — AI Reports & Recommendations

Objective: Generate recurring business intelligence.

- Daily, weekly, and monthly business summaries.
- Trend explanations and recommendations.
- Queue expensive report generation and store historical reports.

PHASE 17 — Knowledge Base & RAG

Objective: Allow businesses to ask questions about their documents.

- Support PDF, DOCX, TXT, and CSV.
- Extract and chunk content, generate embeddings, store vectors, retrieve relevant context, and enforce tenant isolation.

PHASE 18 — AI Usage & Cost Management

Objective: Track and control AI costs.

- Track organization, user, provider, model, input/output tokens, estimated cost, request type, and timestamp.
- Display usage and limits and enforce subscription-based AI limits.

PHASE 19 — Audit Logs & Security

Objective: Harden the application.

- Track who did what, to which resource, when, IP, and user agent.
- Implement rate limiting, secure auth, tenant isolation, validation, mass-assignment protection, secure files, webhook verification, and secret management.

PHASE 20 — Testing

Objective: Make the system reliable.

- Unit tests for calculations, actions/services, permission logic, and AI tool handlers.
- Feature tests for auth, APIs, payments, subscriptions, webhooks, and tenant isolation.
- Explicitly test that Organization A cannot access Organization B.

PHASE 21 — Performance Optimization

Objective: Practice real-world performance engineering.

- Indexes, eager loading, N+1 prevention, reporting query optimization, Redis caching, pagination, and async processing.
- Profile first; do not optimize blindly.

PHASE 22 — Docker & Local Infrastructure

Objective: Make the project reproducible.

- Containerize application, web server, database, Redis, workers, and Node/Next.js as appropriate.
- Document environment variables and local setup.

PHASE 23 — CI/CD

Objective: Automate quality and deployment.

- Install dependencies, run static checks and tests, build frontend, build/deploy application.
- Fail the pipeline when tests fail.

PHASE 24 — Production Deployment

Objective: Deploy on Linux.

- Configure Nginx, PHP-FPM, Laravel, Next.js/Node, Redis, workers, Horizon, scheduler, Reverb, SSL, storage, backups, and environment variables.
- Document deployment and rollback procedures.

PHASE 25 — Monitoring & Observability

Objective: Operate the SaaS like production.

- Monitor application errors, failed jobs, queue health, API latency, database performance, server resources, AI failures, and payment failures.
- Integrate error monitoring such as Sentry and create critical alerts.

6. Core Database Structure

```
users
organizations
organization_users
roles
permissions
role_permissions

customers
categories
products
orders
order_items
payments
transactions
invoices

subscription_plans
subscriptions

notifications
audit_logs

ai_conversations
ai_messages
ai_usage
ai_documents
ai_document_chunks
```

7. API Versioning

Use /api/v1/... from the beginning so future API versions can coexist.

```
GET    /api/v1/customers
POST   /api/v1/customers
GET    /api/v1/customers/{customer}
PUT    /api/v1/customers/{customer}
DELETE /api/v1/customers/{customer}

GET    /api/v1/products
GET    /api/v1/orders
GET    /api/v1/reports
POST   /api/v1/ai/chat
```

8. API Response Standard

```
{
  "success": true,
  "message": "Customer created successfully.",
  "data": {}
}
```

```
{
  "success": false,
  "message": "Unable to create customer.",
  "errors": {}
}
```

9. AI Architecture Rules

- AI must not directly query the database.
- Sensitive business calculations must happen in Laravel.

- Send only the minimum required business data to the model.
- Every AI request must run within an organization context.
- Track AI usage and cost.
- AI failures must not break core business operations.
- Use queues for expensive AI operations where appropriate.

10. Example AI Workflow

```
User:
"Compare my sales this month with last month."

        ↓
AI Assistant
        ↓
Tool Selection
        ↓
compare_sales(current_period, previous_period)
        ↓
Laravel calculates values
        ↓
Structured result
        ↓
AI explains the result
```

11. Recommended Next.js Structure

```
src/
■■■ app/
■■■ components/
■■■ features/
■   ■■■ auth/
■   ■■■ dashboard/
■   ■■■ customers/
■   ■■■ products/
■   ■■■ orders/
■   ■■■ billing/
■   ■■■ reports/
■   ■■■ ai/
■■■ hooks/
■■■ lib/
■■■ services/
■■■ types/
■■■ utils/
```

12. Development Milestones

Milestone	Scope
1	Foundation, authentication, organizations, users, roles, permissions
2	Filament, customers, products, categories, orders
3	Payments, invoices, transactions, subscriptions, Stripe
4	Redis, queues, Horizon, notifications, Reverb
5	Analytics, reports, dashboard
6	AI Assistant, tool calling, AI reports, AI usage
7	Knowledge Base, RAG, embeddings, document processing
8	Testing, security, performance, Docker, CI/CD, deployment, monitoring

13. Learning Objectives

Laravel: Dependency Injection, Service Container, middleware, policies, events/listeners, jobs/queues, notifications, scheduling, API Resources, Form Requests, transactions, Eloquent optimization, caching.

Architecture: modular design, multi-tenancy, separation of concerns, API architecture, event-driven workflows, asynchronous processing.

Database: relationships, constraints, indexes, transactions, aggregation, query optimization.

Infrastructure: Linux, Nginx, Docker, Redis, workers, CI/CD, SSL, monitoring.

AI: LLM APIs, structured outputs, tool calling, embeddings, vector search, RAG, AI cost management.

14. Definition of Done

A feature is not complete merely because it works manually.

- Feature implemented
- Validation added
- Authorization verified
- Tenant isolation verified
- Error handling added
- Tests written
- API documentation updated
- Logging added where required

Production-critical features should pass feature tests, security review, performance review, and deployment verification.

15. Final Development Order

```
1. Laravel 13 fundamentals
2. Clean API architecture
3. Multi-tenancy
4. Authentication & authorization
5. Filament
6. Next.js + API integration
7. Orders & transactions
8. Redis & queues
9. Events & Reverb
10. Stripe & webhooks
11. Analytics
12. AI tool calling
13. RAG
14. Testing
15. Docker
16. CI/CD
17. Production deployment
18. Monitoring & optimization
```

16. Key Principle

Do not start with the AI chatbot. First build a reliable business system, then make AI understand and operate on controlled business data. This produces a much stronger learning and portfolio project.